UNIVERSIDAD POLITÉCNICA DE ATLAUTLA

ACTIVIDAD 2.1 Y 2.2

Problemas y espacios de búsqueda

Modelado formal, grafos y algoritmos en Python

Programa: Ingeniería en Tecnologías de la Información e Innovación Digital
Asignatura: Fundamentos de Inteligencia Artificial
Docente: Javier García Lara
Grupo: 171
Estudiante: Sánchez Valero Crhystian Samuel_____

Introducción

Un problema de búsqueda en inteligencia artificial consiste en encontrar una secuencia de acciones que lleve desde un estado inicial hasta un estado objetivo. Para describirlo se definen los estados, las acciones permitidas, el modelo de transición, el costo y la prueba de meta. Un espacio de búsqueda puede representarse como un grafo: los nodos representan estados y las aristas representan transiciones.

Se modelan dos situaciones: una ruta de entrega entre localidades y la navegación de un robot en una cuadrícula con obstáculos. El primer problema utiliza Dijkstra para

minimizar kilómetros; el segundo utiliza búsqueda en anchura (BFS) para minimizar movimientos.

1. Conceptos fundamentales

Estado: situación actual. Estado inicial: punto de partida. Acciones: operaciones permitidas. Transición: resultado de aplicar una acción. Prueba de meta: comprueba si se alcanzó el objetivo. Costo: suma del costo de las acciones. Un grafo dirigido distingue la dirección de las conexiones; un grafo ponderado asigna un peso a cada conexión.

2. Problema 1: ruta de entrega

Situación: un repartidor sale del Almacén y debe llegar al Hospital. Las conexiones representan tramos de ruta y el peso indica kilómetros. El objetivo es encontrar la ruta de menor distancia.

Formulación formal

Componente	Definición
Estados	Almacén, Centro, Norte, Parque y Hospital.
Estado inicial	Almacén.
Meta	Hospital.
Acciones	Viajar a una localidad conectada por una arista.
Transición	El estado pasa a la localidad de destino.
Costo	Kilómetros del tramo; el costo total suma todos los tramos.

Prueba de meta	El nodo actual es Hospital.

Conexiones y solución

Almacén → Centro (4 km), Almacén → Norte (7 km), Centro → Hospital (5 km), Centro → Parque (3 km), Norte → Parque (2 km), Norte → Hospital (8 km) y Parque → Hospital (2 km). Dijkstra encuentra Almacén → Centro → Parque → Hospital, con costo total de 4 + 3 + 2 = 9 km. Dijkstra es adecuado porque los pesos son no negativos y se desea minimizar la distancia total.

3. Problema 2: robot en una cuadrícula

Situación: un robot se desplaza desde la celda superior izquierda de una cuadrícula de 4 × 4 hasta la inferior derecha. Las celdas (1, 1) y (2, 1) son obstáculos. El objetivo es hallar una ruta con el menor número de movimientos.

Formulación formal

Componente	Definición
Estados	Coordenadas (fila, columna) de cada celda libre.
Estado inicial	(0, 0).
Meta	(3, 3).
Acciones	Mover arriba, abajo, izquierda o derecha, dentro de la cuadrícula y evitando obstáculos.
Costo	Cada movimiento cuesta 1; el costo total es el número de movimientos.

Prueba de meta	La posición actual es (3, 3).

Espacio de búsqueda

Cada celda libre es un nodo y cada movimiento legal es una arista. Los obstáculos no se agregan al grafo. Se utiliza BFS porque todos los movimientos cuestan una unidad. BFS explora primero los estados a una distancia de un paso, después los de dos pasos, etc., por lo que encuentra un camino con el mínimo número de movimientos.

4. Código Python y ejecución

El archivo adjunto actividad_2_1_busqueda.py contiene el programa comentado. Crea ambos grafos, imprime las rutas y genera los diagramas *grafo_rutas_entrega.png* y *grafo_robot.png*.

Instalar dependencias y ejecutar desde la terminal:

```
python -m pip install networkx

matplotlib\npython

actividad_2_1_busqueda.py
```

Los PNG aparecerán en la misma carpeta que el script. Adjunta también las imágenes al repositorio o insértalas en el informe después de ejecutar el código.

5. Análisis de complejidad

Dijkstra: con cola de prioridad, su complejidad típica es O((V + E) log V), donde V es el número de nodos y E el número de aristas.

El almacenamiento del grafo es O(V + E).

BFS: visita cada nodo como máximo una vez y examina cada arista como máximo una vez. Su complejidad temporal es O(V + E) y el espacio auxiliar es O(V). En una cuadrícula de R filas y C columnas, la generación y búsqueda son O(RC), pues cada celda tiene como máximo cuatro vecinos.

6. Conclusiones

El modelado formal convierte situaciones reales en problemas que un algoritmo puede resolver. En la entrega, los estados son localidades y los costos son kilómetros, por lo que Dijkstra selecciona la ruta de menor distancia. En la navegación del robot, los estados son coordenadas y todos los movimientos cuestan lo mismo, por lo que BFS encuentra una ruta mínima en cantidad de pasos.

La elección del algoritmo depende de los costos y de las reglas del problema. Definir correctamente estados, transiciones, obstáculos y metas es esencial: una representación incorrecta puede producir una solución óptima para un modelo que no refleja la realidad. NetworkX facilita construir y visualizar los espacios de búsqueda.

7. Referencias

Russell, S., & Norvig, P. (2020). *Artificial Intelligence: A Modern Approach* (4th ed.). Pearson.

Bohm, R. A. (2021). *The Fundamentals of Search Algorithms*. Nova Science. (Referencia incluida en la guía; verificar datos editoriales con el material de clase).

NetworkX Developers. (s. f.). *NetworkX documentation*. https://networkx.org/documentation/stable/

Python Software Foundation. (s. f.). *Python documentation*. https://docs.python.org/3/